

Construindo o “R” do RAG: um Sistema de Recuperação de Artigos Científicos sobre Bancos de Grafos

Fabio Batista Rodrigues¹

¹Programa de Pós-Graduação em Computação
Faculdade de Computação (FACOM) – UFMS
Campo Grande – MS – Brazil

fabio.batista@ufms.br

Abstract. *This work designs, implements and evaluates a scientific article retrieval system on the topic of graph databases, focused on graph query processing and optimization. The collection (1,097 ArXiv papers) was built to reflect the author’s research project. We implement the two mandatory retrievers, sparse BM25 and a dense retriever based on TF-IDF with cosine similarity, and a hybrid sparse+dense ranking module via Reciprocal Rank Fusion (RRF). Evaluation over 14 test queries, with qrels manually annotated via pooling, shows that the hybrid ranking outperforms both isolated baselines in MAP (0.763 vs. 0.719 for BM25 and 0.695 for TF-IDF) and nDCG@10 (0.797), evidencing the complementarity between the lexical and vector-based strategies.*

Resumo. *Este trabalho projeta, implementa e avalia um sistema de recuperação de artigos científicos sobre o tema de bancos de grafos, com foco em processamento e otimização de consultas em grafos. A coleção (1.097 artigos do ArXiv) foi construída para refletir o projeto de pesquisa. Implementamos dois recuperadores obrigatórios, BM25 esparsa e um recuperador denso baseado em TF-IDF com similaridade do cosseno, e um módulo de aprofundamento de ranking híbrido sparse+dense via Reciprocal Rank Fusion (RRF). A avaliação, sobre 14 consultas de teste com qrels anotados manualmente via pooling, mostra que o ranking híbrido supera ambos os baselines isolados em MAP (0,763 vs. 0,719 do BM25 e 0,695 do TF-IDF) e nDCG@10 (0,797), evidenciando a complementaridade entre as estratégias lexical e vetorial.*

1. Introdução

A explosão na produção científica deslocou o gargalo da pesquisa da escrita para a *descoberta* de trabalhos relevantes. Sistemas modernos de *Retrieval-Augmented Generation* (RAG) [1] dependem de um componente de recuperação, o “R” do RAG, que filtra, antes de qualquer geração de texto, um pequeno conjunto de documentos potencialmente relevantes para a consulta do usuário. Este trabalho tem como objetivo projetar, implementar e avaliar esse componente sobre uma coleção de artigos científicos construída a partir do ArXiv, no domínio de **bancos de grafos**, com foco específico em **processamento e otimização de consultas em grafos**, tema alinhado ao projeto de pesquisa.

O problema de recuperação é tratado, em essência, como um problema de aprendizado de máquina sobre texto: combinamos ponderação probabilística de termos (BM25,

com raízes em Naïve Bayes), busca por vizinhos mais próximos (KNN aplicado à recuperação via TF-IDF e similaridade do cosseno) e fusão de rankings (Reciprocal Rank Fusion) para construir e comparar diferentes estratégias de recuperação.

As contribuições deste trabalho são: (i) uma coleção de 1.097 artigos sobre bancos de grafos, documentada e reproduzível; (ii) dois recuperadores obrigatórios (BM25 e TF-IDF/KNN) implementados sobre a mesma coleção; (iii) um módulo de ranking híbrido (M5) via RRF; e (iv) uma avaliação experimental com 14 consultas e 304 julgamentos de relevância manuais, comparando os três *runs* com P@10, R@10, MAP e nDCG@10.

2. Trabalhos Relacionados

BM25 [2] é o modelo de recuperação esparsa canônico, derivado do *Probabilistic Relevance Framework*, que modela a relevância de um documento como uma probabilidade condicionada à presença de termos da consulta, o mesmo paradigma estatístico subjacente ao Naïve Bayes [3]. Recuperação densa, por outro lado, representa documentos e consultas em um espaço vetorial contínuo; trabalhos como Dense Passage Retrieval [4] e SPECTER [5] mostram que representações aprendidas (*embeddings*) superam TF-IDF quando há variação de vocabulário entre consulta e documento, ao custo de exigirem um modelo pré-treinado. Neste trabalho adotamos TF-IDF como representação densa por ser a baseline recomendada pelo enunciado e por não depender de modelos externos.

A combinação de sinais esparsos e densos (recuperação híbrida) é discutida em Lin [6] como uma visão unificada dos paradigmas de recuperação. Adotamos especificamente o *Reciprocal Rank Fusion* (RRF) [7], que combina rankings por posição em vez de por score, evitando o problema de normalização entre escalas heterogêneas (score BM25 não-normalizado vs. similaridade do cosseno em $[0, 1]$).

Quanto à avaliação, seguimos a metodologia clássica de TREC (consultas + *qrels* + métricas de P@k, R@k, MAP e nDCG), também adotada por benchmarks como o BEIR [8], usado aqui apenas como referência metodológica, não como coleção principal, conforme exigido pelo enunciado.

3. Metodologia

3.1. Escopo da coleção e relação com o projeto de pesquisa

A coleção foi definida a partir do tema de pesquisa “processamento e otimização de consultas em bancos de grafos”. A Tabela 1 resume as decisões de escopo.

A query inicial de coleta (apenas 10 termos centrados em *property graph*/SPARQL e 4 categorias) exauriu em 557 artigos no ArXiv, abaixo do mínimo de 1.000 exigido. Em duas rodadas subsequentes, ampliamos iterativamente palavras-chave (de 10 para 28 termos), categorias (de 4 para 7) e janela temporal (de 2015–2026 para 2008–2026), até atingir 1.097 artigos. Essa iteração ilustra um *trade-off* central da etapa de coleta: termos muito específicos garantem alta precisão temática mas podem não cobrir o volume mínimo exigido pelo enunciado.

A distribuição temporal mostra crescimento acentuado a partir de 2023 (104, 104, 199 e 161 artigos em 2023–2026, respectivamente), consistente com o aumento de interesse em bancos de grafos impulsionado por aplicações de *knowledge graphs* e RAG. A categoria primária predominante é cs.DB (465 artigos), seguida por cs.DC, cs.LG, cs.AI

Tabela 1. Escopo de coleta da coleção.

Campo	Decisão
Palavras-chave	graph database, graph query optimization/processing, property graph, subgraph matching, SPARQL query optimization/processing, Cypher query language, graph indexing, distributed graph query processing, graph processing system, RDF query processing, triple store, graph traversal, join order optimization, query plan optimization, graph data management, graph store, knowledge graph query/storage, graph database benchmark, graph reachability query, regular path query, shortest path query, in-memory graph database, Gremlin query language, graph OLTP
Categorias ArXiv	cs.DB (principal), cs.AI, cs.DC, cs.DS, cs.IR, cs.LG, cs.SE
Janela temporal	2008–2026
Tamanho final	1.097 artigos

e cs.DS, compatível com a natureza interdisciplinar do tema (sistemas de banco de dados, algoritmos distribuídos e, mais recentemente, otimizadores aprendidos).

3.2. Pré-processamento textual

O texto indexado é a concatenação de título e abstract de cada artigo. Aplicamos, de forma idêntica a documentos e consultas (módulo `src/preprocessing.py`): *lower-casing*, remoção de pontuação, tokenização por espaços, remoção de *stopwords* (lista do NLTK) e *stemming* via Porter Stemmer. O *stemming* foi incluído para reduzir variações morfológicas comuns em textos técnicos (e.g., *querying/queries/query* → *queri*), o que aumenta a sobreposição de termos entre consulta e documento tanto para o BM25 quanto para o TF-IDF. O custo dessa escolha é a perda de alguma especificidade lexical (e.g., *indexing* e *indices* colapsam no mesmo *stem*), o que discutimos na Seção 4.

3.3. Recuperação esparsa, BM25

Implementado com a biblioteca `rank_bm25` (BM25Okapi) sobre o texto pré-processado de título+abstract. Adotamos $k_1 = 1,5$ e $b = 0,75$. $k_1 = 1,5$ está dentro da faixa típica (1,2–2,0); como abstracts científicos são curtos e raramente repetem termos excessivamente, não há necessidade de saturar a contribuição de um termo mais rapidamente (o que exigiria k_1 menor). $b = 0,75$ é o padrão da literatura [2] e é adequado pois os documentos da coleção (título+abstract) têm comprimento relativamente homogêneo. O BM25 modela a relevância como uma função da frequência do termo no documento (TF) ponderada

pela raridade do termo no corpus (IDF), o mesmo paradigma probabilístico, com a mesma suposição de independência entre termos, do Naïve Bayes visto na disciplina.

3.4. Recuperação por vizinhos mais próximos, TF-IDF/KNN

Cada documento e cada consulta são representados como vetores TF-IDF (`TfidfVectorizer` do `scikit-learn`, `min_df=2`, `max_df=0,85`), ajustados sobre o vocabulário da coleção. O ranking é dado pelos K documentos com maior similaridade do cosseno entre o vetor da consulta e os vetores dos documentos, exatamente a lógica do KNN visto em sala, mas devolvendo a lista de vizinhos como resposta em vez de votar uma classe. Optamos por TF-IDF (em vez de *embeddings* pré-treinados) como representação densa por ser a baseline recomendada pelo enunciado e por não introduzir dependência de modelos externos não cobertos pela disciplina.

3.5. Módulo de aprofundamento M5, Ranking híbrido

Combinamos os *runs* de BM25 e TF-IDF/KNN via *Reciprocal Rank Fusion* [7]:

$$\text{RRF}(d) = \sum_{s \in \{\text{bm25}, \text{knn}\}} \frac{1}{\kappa + \text{rank}_s(d)}, \quad (1)$$

com $\kappa = 60$ (valor original do RRF). RRF foi escolhido em vez de soma ponderada de scores porque BM25 (não-normalizado, $[0, \infty)$) e similaridade do cosseno ($[0, 1]$) vivem em escalas incompatíveis, combinar por *posição* no ranking evita o problema de normalização entre paradigmas heterogêneos, sem exigir um hiperparâmetro de peso α adicional.

3.6. Avaliação experimental

Construímos 14 consultas de teste cobrindo subáreas distintas do tema: métodos (sub-graph matching, join order, indexação), linguagens de consulta (Cypher, SPARQL), sistemas distribuídos, *surveys* e *benchmarks* (lista completa em `eval/queries.tsv`).

O *qrels* foi construído via *pooling*: para cada consulta, unimos o top-15 de BM25 e o top-15 de TF-IDF/KNN (com sobreposição parcial), totalizando 304 pares (consulta, documento) anotados com relevância binária (0/1). O critério de relevância adotado foi: *um documento é relevante se seu título/abstract trata diretamente do método, sistema ou survey que a consulta busca*. Do total, 173 julgamentos (56,9%) foram marcados como relevantes. Reportamos P@10, R@10, MAP e nDCG@10 com o script `eval/evaluate.py` fornecido pela disciplina.

Limitação: como o *qrels* foi construído via *pooling* limitado ao top-15 de dois sistemas, R@10 é uma estimativa, documentos relevantes fora desse pool não são contabilizados, favorecendo levemente os próprios sistemas que geraram o pool.

4. Resultados e Discussão

O ranking híbrido supera ambos os baselines isolados em todas as métricas, com ganho mais expressivo em MAP (+6,1% sobre BM25, +9,7% sobre TF-IDF). Isso confirma a complementaridade entre os paradigmas: BM25 vence em consultas com vocabulário técnico discriminativo e baixa ambiguidade lexical (e.g., termos exatos como “Cypher”, “SPARQL”), enquanto o TF-IDF/KNN tende a vencer quando a consulta usa fraseado

Tabela 2. Métricas médias sobre as 14 queries de teste.

Sistema	P@10	R@10	MAP	nDCG@10
BM25	0,657	0,577	0,719	0,794
TF-IDF/KNN	0,671	0,565	0,695	0,764
Híbrido (M5)	0,671	0,580	0,763	0,797

mais genérico que casa com múltiplos documentos relacionados, mesmo sem repetição exata de termos raros.

Estudo qualitativo, acerto (q1). Para a consulta “*subgraph matching algorithms for graph databases*”, ambos os sistemas atingem $P@10 = 1,0$ e $AP > 0,88$. O pool de candidatos para essa consulta foi quase inteiramente composto por artigos diretamente sobre *subgraph matching*, um caso em que o vocabulário da consulta coincide quase exatamente com o vocabulário técnico da subárea, favorecendo tanto a ponderação de termos (BM25) quanto a similaridade vetorial (TF-IDF).

Estudo qualitativo, falha relativa (q11). Para “*learned query optimizers for graph databases*”, o BM25 obtém $AP = 0,823$ enquanto o TF-IDF/KNN cai para $AP = 0,233$. A consulta combina um conceito específico (otimizadores *aprendidos*, i.e., baseados em aprendizado de máquina) com um domínio amplo (bancos de grafos); o TF-IDF tende a recuperar artigos genéricos sobre otimização de consultas ou sobre aprendizado em grafos isoladamente, sem a interseção exata dos dois conceitos, pois a similaridade do cosseno é dominada pelos termos mais frequentes do vocabulário (‘graph’, ‘query’, ‘optimization’), diluindo o sinal mais raro e discriminativo (‘learned’/‘learning’). O BM25, por ponderar termos raros mais fortemente via IDF, consegue capturar melhor essa interseção.

Observação adicional. Na consulta q8 (“*index structures for accelerating graph queries*”), ambos os sistemas têm desempenho moderado ($MAP \approx 0,44\text{--}0,54$) porque o pool mistura artigos sobre indexação de bancos de grafos (relevantes) com artigos sobre índices de busca aproximada de vizinhos (ANN) em espaços vetoriais de alta dimensão (não relevantes ao tema de bancos de grafos, apesar de também usarem a palavra “graph” para descrever a estrutura do índice). Esse caso ilustra um limite de ambos os paradigmas léxico e vetorial clássico: a ambiguidade semântica do termo “graph” (estrutura de dados de armazenamento de relações vs. estrutura de indexação para ANN) não é resolvida nem por contagem de termos nem por TF-IDF, e provavelmente exigiria *embeddings* contextuais ou re-ranqueamento supervisionado (módulo M1) para ser mitigada.

5. Conclusão

Implementamos e avaliamos um sistema de recuperação de artigos científicos sobre bancos de grafos, cobrindo o pipeline obrigatório (coleta, pré-processamento, BM25 e TF-IDF/KNN) e o módulo de aprofundamento M5 (ranking híbrido via RRF). Os resultados mostram que a fusão de sinais esparsos e densos supera ambos os componentes isolados nas quatro métricas avaliadas, confirmando a complementaridade entre os paradigmas estudados na disciplina. Como trabalho futuro, destacam-se: (i) substituir TF-IDF por *embeddings* pré-treinados (e.g., SPECTER2) para mitigar a ambiguidade semântica ob-

servada na consulta q8; (ii) expandir o *qrels* para reduzir o viés do *pooling*; e (iii) explorar o módulo M1 (re-ranqueamento supervisionado) sobre o top-100 do híbrido.

Uso de assistentes de IA generativa

O copilot foi utilizado para gestão do código auxiliando em tarefas repetitivas como atualização de documentação e correção de erros dado uma pré-visualização de erros e entendimento deles.

1

Referências

- [1] Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS*.
- [2] Robertson, S. & Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333–389.
- [3] Manning, C. D., Raghavan, P. & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.
- [4] Karpukhin, V. et al. (2020). Dense Passage Retrieval for Open-Domain Question Answering. *EMNLP*.
- [5] Cohan, A. et al. (2020). SPECTER: Document-level Representation Learning using Citation-informed Transformers. *ACL*.
- [6] Lin, J. (2021). A Proposed Conceptual Framework for a Representational Approach to Information Retrieval. *SIGIR Forum*, 55(2).
- [7] Cormack, G. V., Clarke, C. L. A. & Buettcher, S. (2009). Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. *SIGIR*.
- [8] Thakur, N. et al. (2021). BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models. *NeurIPS Datasets & Benchmarks*.

¹Vídeo de apresentação: <https://...> (adicionar link antes da entrega).